[Khanh Pham] Process Fundamentals, Memory Layout

Process Management, Memory Layout

1. Process management

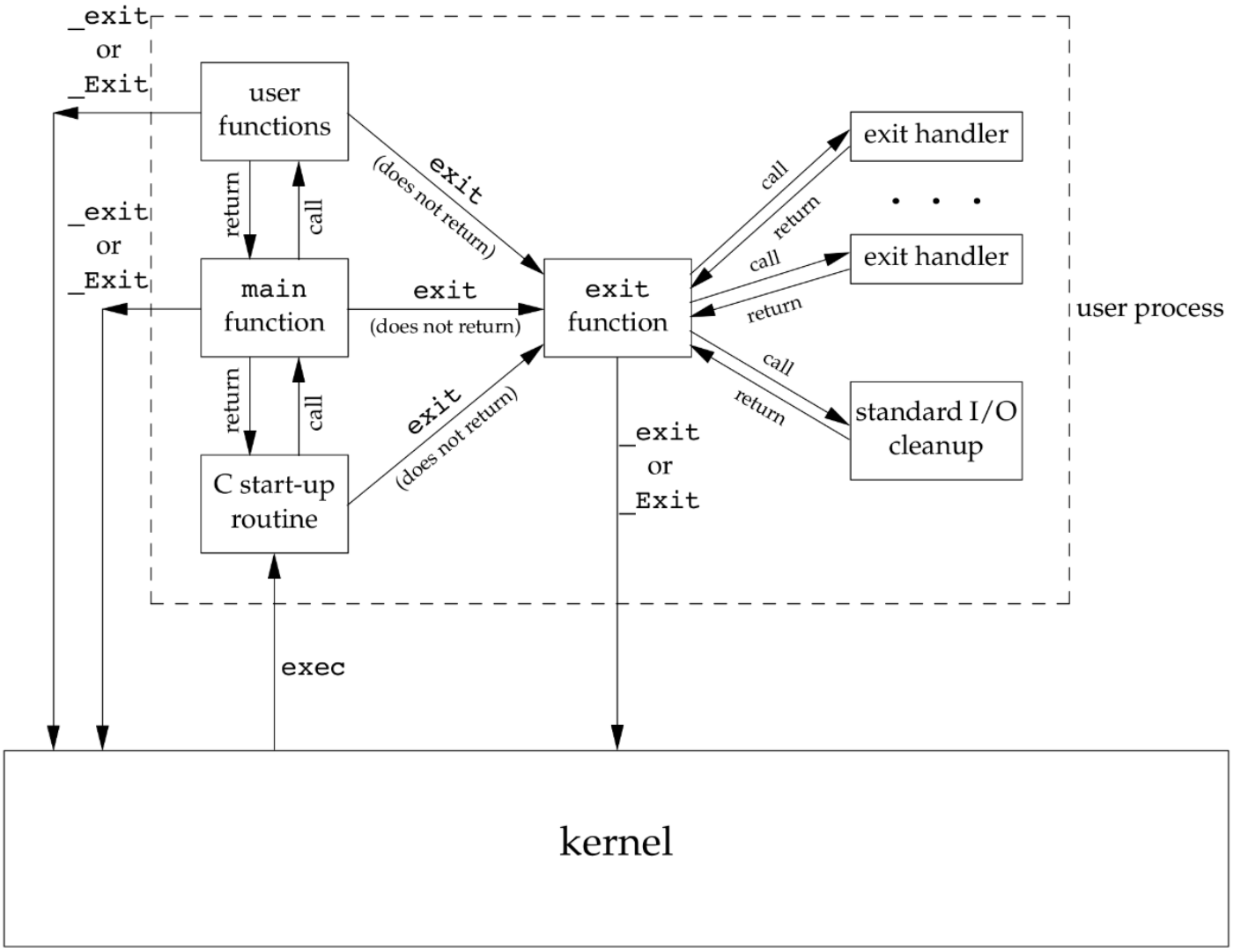
> In this section, I will not repeat basic information about process-related functions, such as the syntax and basic usage of `fork`, `vfork`, `waitpid`, and `system`, since these topics are well documented in Linux books and online resources.
>
> Instead, I will focus on the key differences between functions that can potentially introduce bugs if they are not used carefully, such as `exit`/`_exit`/`_Exit`, `fork`/`vfork`, `wait`/`waitpid`, and `exec`/`system`.
>
> I will also discuss race conditions, explain how they can occur, and present ways to prevent or fix them. In addition, I will cover several situations that can be problematic for programs, such as **zombie processes** and **orphan processes**.

exit and _exit function

As we know, when a program terminates, there are several ways to terminate the process, such as using `exit`, `_exit`, and `return`.

- `exit` terminates the process after performing the necessary cleanup, such as flushing and closing standard I/O streams and calling functions registered with `atexit`.
- `_exit` terminates the process immediately without performing the cleanup handled by `exit`. We can also use `_Exit`; both functions have essentially the same behavior. `_Exit` is defined by the ISO C standard, while `_exit` is the POSIX/Unix interface.
- `return` behaves differently depending on where it is used. When `return` is used in the `main` function, it terminates the process and has behavior equivalent to calling `exit` with the returned status. However, when `return` is used in a normal function, it only terminates that function and returns control to its caller; it does **not** terminate the process.

The relationship between `exit` and `_exit` is illustrated in the picture below:



As a result, when we call `exit` function, that mean we also call all standard cleaning function in C library before calling the `_exit` function.

The bellow example will explain the bug when we use `exit` instead of `return` funtion

```
void action()
{
    printf("do the first action\n");
    printf("do the second action\n");
    return;
}
int main()
{
    action();
    printf("use the result of action function\n");
}
```

The result when we run this program:

```
do the first action
do the second action
use the result of action function
```

However, if we replace the `return` funtion by `exit` funtion, the result can be:

```
do the first action
do the second action
```

As you see, after completing the `action` funtion, the process will be killed and cannot run the next logic code. So we can know when we want the process is no longer running and terminate, we should use `exit` function. Nevertherless, if we want the process still run and do next logic code, we must use `return` funtion.

After understanding how a process terminates, the next important issue is how a new process is created and how parent and child processes interact

Fork and vfork function

Both `fork` and `vfork` are function to create a child process. Both of them return the `0` in child process and the `PID` of child process in parent process. However, there are some different of them:

First, with `fork` function, Parent and child processes are not share the same address space, They look like a 2 process run independently, so we can't predict what process will be run first.

Can we look the example of `fork` function bellow:

```
int main(void)
{
    __pid_t pid = fork();
    if (pid < 0)
    {
        printf("error\n");
    }
    else if (pid == 0)
    {
        for (int i = 0; i < 5; i++)

        {
            printf("child, pid = %d\n", getpid());
        }
    }
    else
    {
        for (int i = 0; i < 5; i++)

        {
            printf("parent, pid = %d\n", getpid());
        }
    }

    exit(0);
}
```

After running the program, the result sometimes look like:

```

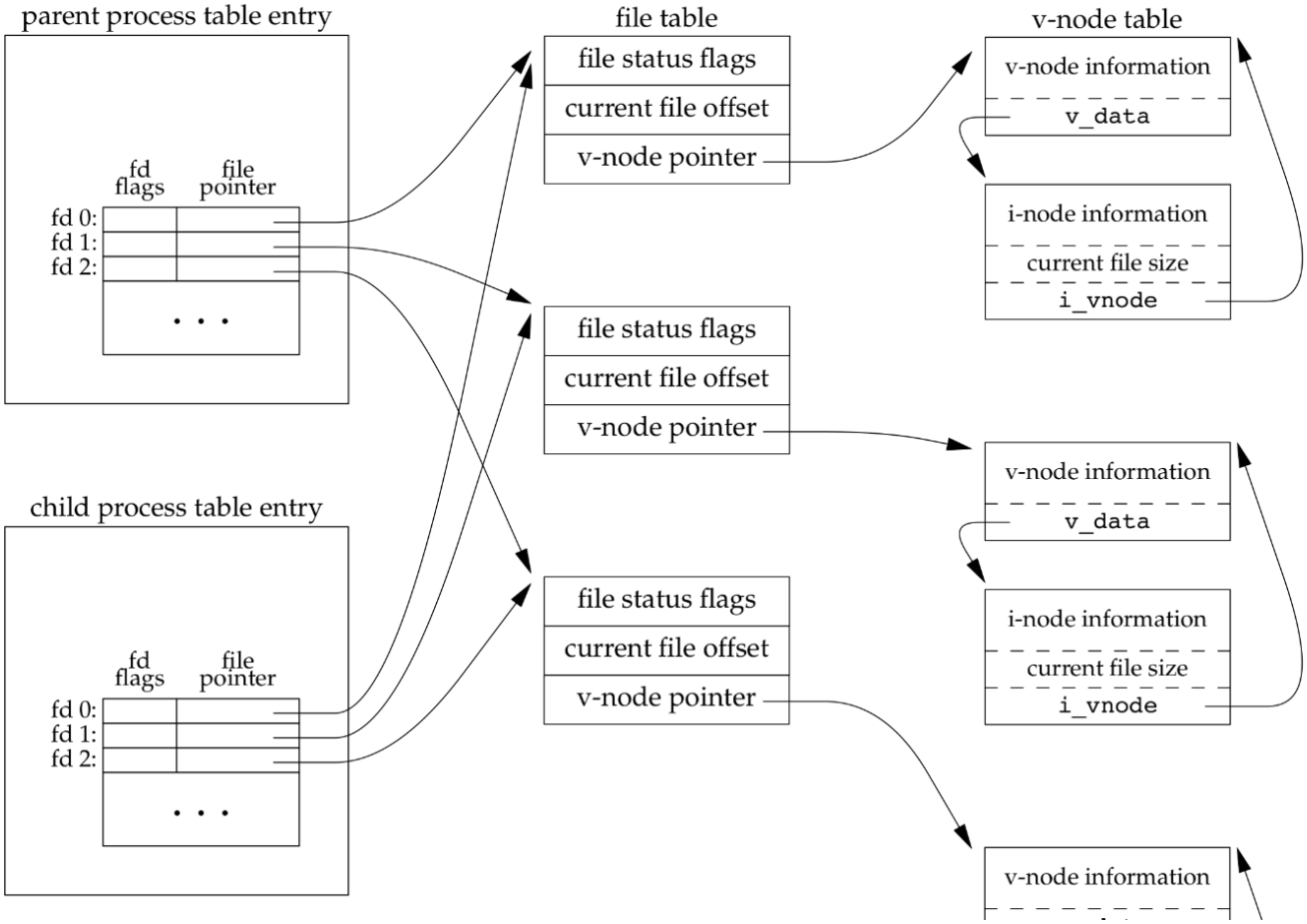

parent, pid = 19983
parent, pid = 19983
child, pid = 19984
parent, pid = 19983
child, pid = 19984
parent, pid = 19983
child, pid = 19984
```

As we can see, normally, we can't predict which process will be chose by CPU schedule. So, how can we do if we want child or parent must run first. Before answering the question, we have to

understand the relationship between parent and child data when we use `fork` function. This is fields that child process copy of parent process:

```
Real UID/GID
Effective UID/GID
Supplementary groups
Process group ID
Session ID
Controlling terminal
Set-user-ID / set-group-ID flags
Current working directory
Root directory
File mode creation mask (umask)
Signal mask
Signal dispositions
Close-on-exec flags
Environment
Shared memory attachments
Memory mappings
Resource limits
```

As we can see, the file description(fd) in field `close-on-exec flags` is copied by child too. However, this fd is just not only a normal number, this fd point exactly to the same file with parent. We can imagine by this picture:



Now return the question, how we can make the child or parent run first. The simple way is we can use a basic IPC: `pipe`. Because both 2 processes have fd refer to the the same file, we can make the parent or child process must wait until having any character which appear in the pipe. See the example:

```
int main()
{
    int fd[2];
    pipe(fd);

    pid_t pid = fork();

    if (pid == 0)
    {
        close(fd[1]);

        char c;
        read(fd[0], &c, 1);

        printf("Child run later\n");
```

```
        close(fd[0]);
    }
    else
    {
        close(fd[0]);

        printf("Parent run first\n");

        write(fd[1], "x", 1);

        close(fd[1]);
        wait(NULL);
    }
}
```

In this example, I make the child process wait until the parent process writes to the pipe. Therefore, the child process has to run first and write to the pipe to notify the parent process when the operation is complete.

With the `vfork()` function, the parent and child processes temporarily share the same address space. In addition, the parent process is suspended until the child process either calls `_exit()` / `_Exit()` or executes an `exec` function. Therefore, in this case, the child process runs before the parent can continue.

Let me explain what **the same address space** means with the following example:

```
int main(void)
{
    int num = 5;
    __pid_t pid = vfork();

    if (pid < 0)
    {
        printf("error\n");
    }
    else if (pid == 0)
    {
        num++;
        printf("%d\n", num);
        _exit(0);
```

```
    }
    else
    {
        printf("%d\n", num);
    }
}
```

In this example, because the parent process is suspended while the child is running, and the child temporarily shares the parent's address space, both processes access the same memory location for the `num` variable.

The child increments `num` from `5` to `6` and then terminates with `_exit()`. When the parent resumes, it reads the value from the same memory location, so the result is:

```
6
6
```

However, what happens if we replace `_exit()` in the child with `return`, or remove it entirely?

This results in **undefined behavior**. The child must not return from the function that called `vfork()` or continue executing code in a way that violates the restrictions of `vfork()`. In particular, returning from `main` can cause the child to execute termination logic that is not valid in this situation.

As a result, the program may produce unexpected output, such as:

```
6
352605712
```

The exact output is not predictable because the behavior is undefined.

Now we understand about child and parent process. The question is how the parent process know when the child process done, we can use the `wait` or `waitpid` that will be dicussed now.

Wait, Waitpid function

When a child process terminates, the kernel sends a `SIGCHLD` signal to its parent process to notify it that the child has changed state. Normally, the parent should call `wait` or `waitpid` to collect the child's termination status and allow the kernel to release the remaining resources associated with the terminated child.

- `wait` blocks the calling process until **one of its child processes** terminates or changes state. It then returns the PID of the child and provides its termination status through the `status`

argument.
- `waitpid` provides more control over which child process to wait for. It can wait for a specific child by PID, a process group, or any child, depending on the value of its `pid` argument. It also supports options that change its behavior. For example, `WNOHANG` makes `waitpid` return immediately if no child has changed state, instead of blocking the caller.

Both `wait` and `waitpid` can be used to **collect (reap) the termination status of a child process**. The status contains information about how the child terminated, such as whether it exited normally and its exit status, or whether it was terminated by a signal. Macros such as `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, and `WTERMSIG` can be used to interpret this status.

It is important to understand that **receiving `SIGCHLD` does not itself remove the terminated child from the process table**. The parent still needs to call `wait` or `waitpid` to reap the child. Until that happens, the terminated child remains as a **zombie process**.

Zombie and Orphan process

Zombie process

Now we will discuss the **process lifecycle**. First, we will look at the situation where a child process terminates but the parent process does not call `wait()` or `waitpid()`. This results in a **zombie process**.

```
int main()
{
    int i;
    int pid = fork();

    if (pid == 0)
    {
        for (i = 0; i < 20; i++)
            printf("Child process\n");
    }
    else
    {
        printf("Parent process\n");
        while (1);
    }

    return 0;
}
```

After the child process finishes, it terminates but is not immediately removed from the process table because the parent has not called `wait()` or `waitpid()` to collect its termination status.

We can see this by checking the process status:

```
12766 pts/0    R+     0:05 ./test
12767 pts/0    Z+     0:00 [test] <defunct>
```

The `Z` state and `<defunct>` indicate that the child is a **zombie process**. The zombie is no longer running, but the kernel keeps some information about it, such as its PID and exit status, until the parent reaps it.

How to prevent zombie processes?

The recommended solution is to call `wait()` or `waitpid()` in the parent process:

```
int status;
waitpid(pid, &status, 0);
```

For programs that create multiple children, the parent should properly reap all terminated children, often by handling `SIGCHLD`.

Another way for the zombie to be eventually reaped is for the parent process to terminate. The orphaned child is then adopted by init process or another configured subreaper, which can reap it later. However, **killing the parent is not a proper solution**; the parent should manage its child processes correctly. Moreover, this situation can be called `orphan` that will be discussed bellow:

Orphan process

As we know, in many cases, when we use `fork()` to create a child process, the parent process uses `wait()` or `waitpid()` to wait for the child to finish. However, what happens if the parent process terminates before the child?

This creates an **orphan process**.

So, after the parent process terminates, what happens to the child?

In most cases, the orphan process is **reparented to the `init` process (usually `systemd`) with PID 1**. The new parent can then take care of the child and reap it when the child eventually terminates.

As we can see in the example below:

```
int main()
{
```

```
    int pid = fork();
    if (pid == 0)
    {
        for (int i = 0; i < 100; i++)
        {
            printf("child: pid = %d, ppid = %d\n", getpid(), getppid());
            sleep(1);
        }
    }
    else
    {
        for (int i = 0; i < 5; i++)
        {
            printf("parent: pid = %d\n", getpid());
            sleep(1);
        }
        exit(0);
    }
}
```

According to theory, when the parent process dies, the child process will reparent the process which `pid = 1`. However, the real result is:

```
parent: pid = 10199
child: pid = 10200, ppid = 10199
parent: pid = 10199
child: pid = 10200, ppid = 10199
parent: pid = 10199
child: pid = 10200, ppid = 10199
child: pid = 10200, ppid = 10199
parent: pid = 10199
parent: pid = 10199
child: pid = 10200, ppid = 10199
child: pid = 10200, ppid = 10199
child: pid = 10200, ppid = 1431/* this line marks the parent process died */
child: pid = 10200, ppid = 1431
child: pid = 10200, ppid = 1431
child: pid = 10200, ppid = 1431
child: pid = 10200, ppid = 1431
```

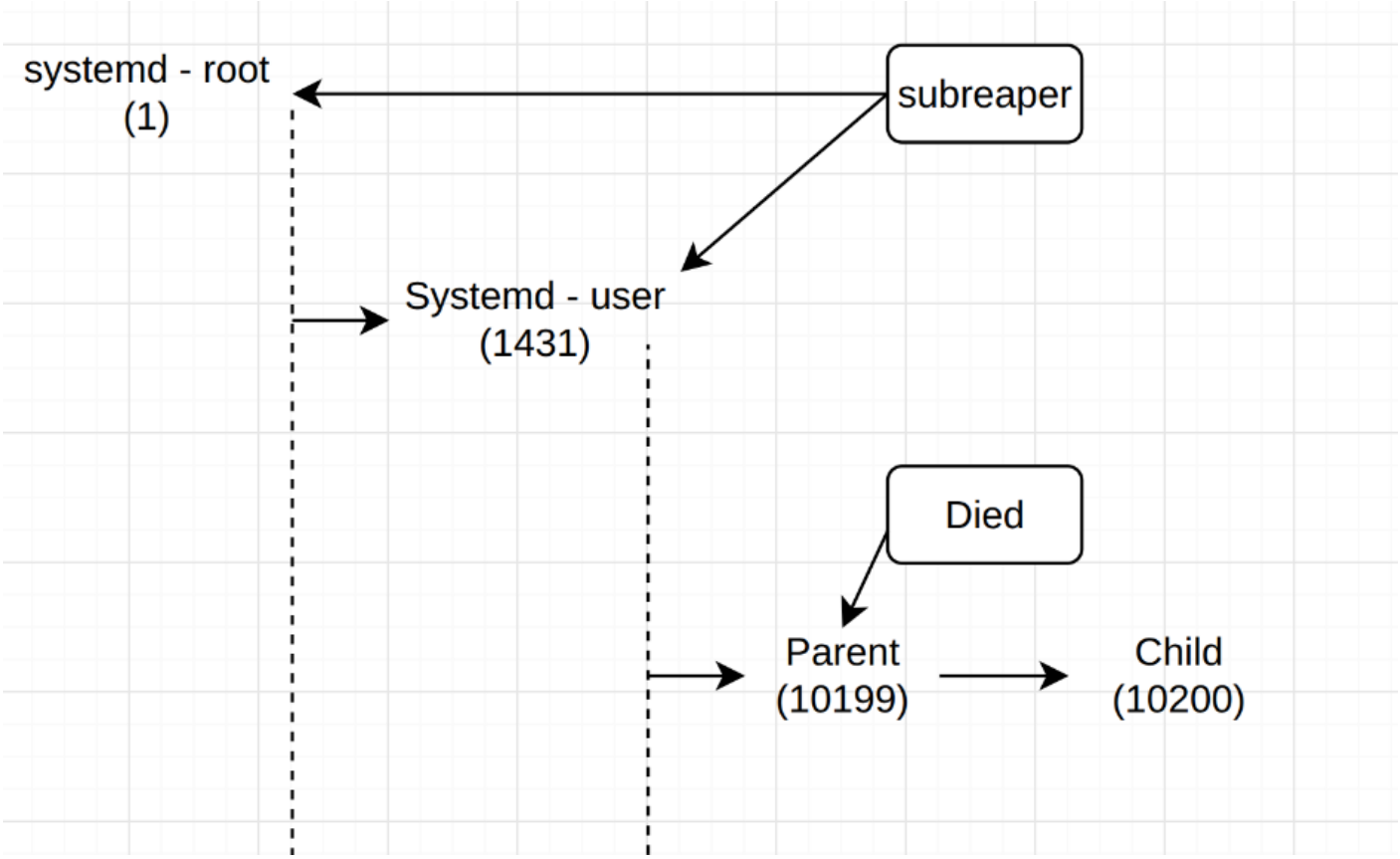
```
child: pid = 10200, ppid = 1431
child: pid = 10200, ppid = 1431
child: pid = 10200, ppid = 1431
child: pid = 10200, ppid = 1431
```

So, what is `ppid = 1431`, why not `ppid = 1` like the theory?

Since version 3.4, Linux has had the concept of a subreaper. As the definition in man7: `prctl`:

> A subreaper fulfills the role of init(1) for its descendant processes. When a process becomes orphaned (i.e., its immediate parent terminates), then that process will be reparented to the nearest still living ancestor subreaper. Subsequently, calls to getppid(2) in the orphaned process will now return the PID of the subreaper process, and when the orphan terminates, it is the subreaper process that will receive a SIGCHLD signal and will be able to wait(2) on the process to discover its termination status.

So the relationship between processes in the example above can see like that:



Now let's try a sample with `prctl` function. As a example bellow, I will create 3 processes that have a relationship: `parent - child - grandchild`. Besides, i will use `prctl` function to make the `parent` process become the nearest subreaper process. After that, i will make the `child` process die and

the `grandchild` process become orphan.

```
int main()
{
    if (prctl(PR_SET_CHILD_SUBREAPER, 1) == -1)
    {
        perror("prctl");
        return 1;
    }

    printf("parent: PID=%d, PPID=%d\n",
           getpid(), getppid());

    pid_t child = fork();

    if (child < 0)
    {
        perror("fork");
        return 1;
    }

    if (child == 0) /*child*/
    {
        pid_t grandchild = fork();

        if (grandchild < 0)
        {
            perror("fork");
            exit(1);
        }

        if (grandchild == 0) /*grandchild*/
        {
            for (int i = 0; i < 100; i++)
            {
                printf("Grandchild: PID=%d, PPID=%d\n",
                       getpid(), getppid());
                sleep(1);
            }
        }
```

```
        else
        {
            for (int i = 0; i < 5; i++) /*child*/
            {
                printf("child, pid = %d, ppid = %d\n", getpid(), getppid());
                sleep(1);
            }

            exit(0);
        }
    }
    else /*parent*/
    {
        while (1)
            ;
    }

    return 0;
}
```

The result is here: Now, when the grandchild becomes orphan, it reparents to the parent process

```
parent: PID=10045, PPID=3494
child, pid = 10046, ppid = 10045
Grandchild: PID=10047, PPID=10046
child, pid = 10046, ppid = 10045
Grandchild: PID=10047, PPID=10046
child, pid = 10046, ppid = 10045
Grandchild: PID=10047, PPID=10046
child, pid = 10046, ppid = 10045
Grandchild: PID=10047, PPID=10046
child, pid = 10046, ppid = 10045
Grandchild: PID=10047, PPID=10046
Grandchild: PID=10047, PPID=10046
Grandchild: PID=10047, PPID=10045
Grandchild: PID=10047, PPID=10045
```

> **Why we need the supreaper process?** Establishing a subreaper process is useful in session management frameworks where a hierarchical group of

> processes is managed by a subreaper process that needs to be informed when one of the processes—for example, a double-forked daemon—terminates (perhaps so that it can restart that process).

After understanding the **orphan process**, we can move on to another important topic: how a process can execute another program.

Exec and system function

Exec function

So far, we have discussed how processes are created, terminated, and managed. Now, we will look at two commonly used functions for executing programs: `exec` and `system`.

Although both are used to execute another program or command, they work differently. Understanding these differences is important because using the wrong function can lead to unexpected behavior or security issues.

> `exec` **family:** `exec` isn't only a function, it has a family with many types like : `execl`,`execv``execve`,,`execle`,`execlp`,`execvp`

The `exec` family replaces the **current process image** with a new program. It does not create a new process.

For example:

```
int main()
{
    printf("Before exec\n");

    execl("/bin/ls", "ls", "-l", NULL);

    printf("After exec\n");

    return 0;
}
```

If `execl()` succeeds, the original program is completely replaced by `ls`, so:

```
Before exec
<output of ls>
```

The `After exec` message will **not** be printed because the original process image no longer exists.

If `exec` fails, it returns `-1`, and the original process continues executing.

This allows the parent and child to run different programs.

System function

Unlike `exec()`, `system()` does not replace the current process. It executes a command through a shell and waits for the command to finish. Inside the system function, it will create a child process and run the exec function in shell:

```
int system(const char *cmdstring)
{
    pid_t
        pid;
    int
        status;
    if (cmdstring == NULL)
        return (1);
    if ((pid = fork()) < 0)
    {
        status = -1;
    }
    else if (pid == 0)
    {
        /* child */
        execl("/bin/sh", "sh", "-c", cmdstring, (char *)0);
        _exit(127);
    }
    else
    {
        while (waitpid(pid, &status, 0) < 0)
        {
            if (errno != EINTR)
            {
                status = -1; /* error other than EINTR from waitpid() */
                break;
```

```
            }
        }
    }
    return (status);
}
```

For example:

```
int main()
{
    printf("Before system\n");

    system("ls -l");

    printf("After system\n");

    return 0;
}
```

The result look like:

```
Before system
<output of ls>
After system
```

Therefore, `system()` is convenient when we want to execute a shell command directly, while `exec()` provides more control over which program is executed.

The dangerous action when using `system` function

One important difference is that `system()` invokes a shell, so using untrusted input to construct the command can introduce **command injection vulnerabilities**:

```
system(user_input);
```

Look the example:

```
int main()
{
    char filename[100];

    printf("File name:  ");
```

```
    scanf("%99s", filename);

    char cmd[150];
    snprintf(cmd, sizeof(cmd), "cat %s", filename);

    system(cmd);
}
```

When we use the `system()` function, it behaves similarly to typing a command directly into a shell. For example, if the input is:

```
hello.txt; ls
```

the shell interprets `;` as a command separator. Therefore, it executes two commands instead of treating the entire input as a filename:

```
cat hello.txt
ls
```

The result may look like:

```
File name: hello.txt;ls
cat: hello.txt: No such file or directory
add.c  add.o  singlyLinkList.c  sub.o  test2.c  test.txt
add.i  singlyLinkList  sub.c  test  test.c
```

This becomes a security problem if the user can provide arbitrary input and the input contains a malicious command.

Now, if we replace `system()` with `exec`, for example:

```
execlp("cat", "cat", filename, NULL);
```

and enter the same input:

```
hello.txt;ls
```

the result is:

```
File name: hello.txt;ls
cat: 'hello.txt;ls': No such file or directory
```

In this case, `exec` passes `hello.txt;ls` directly to `cat` as an argument. It does not invoke a shell, so `;` is not interpreted as a command separator.

Therefore, when executing a program with user-controlled input, using the appropriate `exec` function is generally safer than using `system()`, because `exec` does not interpret the input as shell commands.

Besides the security risks associated with `system()`, another common problem when working with multiple processes is a **race condition**.

Race Condition

A race condition occurs when multiple processes access shared data concurrently, and the final result depends on the order in which the processes execute.

When working with processes, race conditions commonly occur when using **shared memory IPC**. For example:

```
int main(void)
{
    int *counter = mmap(
        NULL,
        sizeof(int),
        PROT_READ | PROT_WRITE,
        MAP_SHARED | MAP_ANONYMOUS,
        -1,
        0);

    *counter = 0;

    pid_t pid = fork();

    if (pid == 0)
    {
        for (int i = 0; i < 100000; i++)
            (*counter)++;

        _exit(0);
    }
    else
    {
```

```
        for (int i = 0; i < 100000; i++)
            (*counter)++;

        wait(NULL);

        printf("counter = %d\n", *counter);
    }
}
```

As we can see, both children and parent process use the same memory and each increase the `counter` variable 100000 times. However, the result is not always 2000000 like this:

```
khanhnp@khanhnp-17ZD90N-VX50K:~/C_programming$ ./test
counter = 122709
khanhnp@khanhnp-17ZD90N-VX50K:~/C_programming$ ./test
counter = 118801
```

> In this situation, both the parent and child processes increase the `counter` without considering the current value of the variable. As a result, both processes may read the same value of `counter` and then increase it at the same time.
>
> Therefore, even though there are two increment operations, the value of `counter` may increase by only **1** instead of **2**. This is a typical example of a **race condition**.

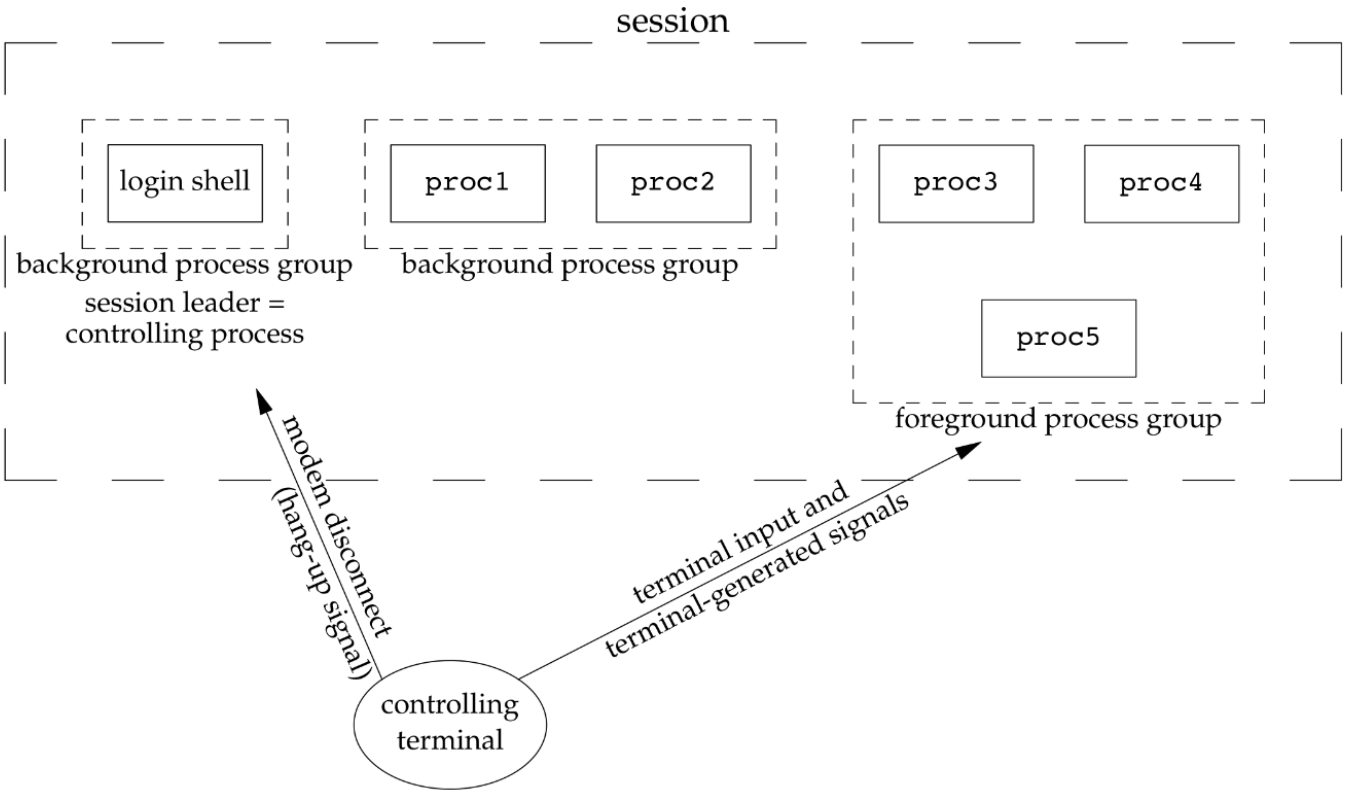
Process group, session, terminal controlling and job control

Process group is a colection of one or more processes. Each process group has unique `PGID`. Each process group has process group leader that have the same `PID` with `PGID`.

A session is the collection of one or more processes. There are two types of process group:

- Foreground process group: This process group can interact in bash
- Background process group: This process groupd cannot interact in bash

A process group calles a job. A session can have a single controlling terminal. We can imagine by this picture:



Job control is the feature that allows us to start multiple jobs (groups of processes) from a single terminal and to control which jobs can access the terminal and which jobs are run in the background. Each terminal has only 1 foregroup process group. Some signal that is useful:

- `ctrl + C`: This action will send the `SIGINT` signal to the foregroup process in this terminal
- `ctrl + Z`: This action will send `SIGSTP` to the foregroup process group in terminal. After that, we can use the command `fg` to continue the stoped process in this terminal. The command `bg` to continue the stoped process group in backgroup.

Daemon

A daemon is a process with some special features:

- It is long-lived. Often, a daemon is created at system startup and runs until the system is shut down
- It runs in the background and has no controlling terminal.

Steps to make a daemon:

- **fork** off the parent process & let it terminate if forking was successful. -> Because the parent process has terminated, the child process now runs in the background.
- **setsid** - Create a new session. The calling process becomes the leader of the new session and the process group leader of the new process group. The process is now detached from its controlling terminal (CTTY).
- **Catch signals** - Ignore and/or handle signals.

- **fork again** & let the parent process terminate to ensure that you get rid of the session leading process. (Only session leaders may get a TTY again.)
- **chdir** - Change the working directory of the daemon.
- **umask** - Change the file mode mask according to the needs of the daemon.
- **close** - Close all open file descriptors that may be inherited from the parent process and redict them to `/dev/null`

We can see the skeleton of daemon source code here:

```
// Global volatile flag for clean signal-driven shutdown
volatile sig_atomic_t keep_running = 1;

/**
* Signal handler for graceful daemon termination.
*/
void handle_signal(int sig) {
   if (sig == SIGTERM || sig == SIGINT) {
       keep_running = 0;
   }
}

/**
* Transforms the current process into a background daemon.
*/
static void skeleton_daemon(void) {
   pid_t pid;

   /* 1. First fork: Detaches the child process from the parent terminal */
   pid = fork();
   if (pid < 0) {
       exit(EXIT_FAILURE); // Fork failed
   }
   if (pid > 0) {
       exit(EXIT_SUCCESS); // Terminate parent process
   }

   /* 2. Create a new session and process group (becomes Session Leader) */
   if (setsid() < 0) {
       exit(EXIT_FAILURE);
   }
```

```
/* 3. Handle signal masks */
signal(SIGCHLD, SIG_IGN); // Ignore child termination signals
signal(SIGHUP, SIG_IGN);  // Ignore terminal hangup signals
signal(SIGTERM, handle_signal); // Catch termination signal
signal(SIGINT, handle_signal);  // Catch interrupt signal

/* 4. Second fork: Relinquish Session Leadership to prevent re-attaching to TTY */
pid = fork();
if (pid < 0) {
    exit(EXIT_FAILURE);
}
if (pid > 0) {
    exit(EXIT_SUCCESS); // Second parent terminates
}

/* 5. Set permissive file creation mask */
umask(0);

/* 6. Change working directory to root to prevent locking mounted filesystems */
if (chdir("/") < 0) {
    exit(EXIT_FAILURE);
}

/* 7. Close standard file descriptors and redirect them to /dev/null */
close(STDIN_FILENO);
close(STDOUT_FILENO);
close(STDERR_FILENO);

int dev_null = open("/dev/null", O_RDWR);
if (dev_null != -1) {
    dup2(dev_null, STDIN_FILENO);  // Standard Input  (FD 0)
    dup2(dev_null, STDOUT_FILENO); // Standard Output (FD 1)
    dup2(dev_null, STDERR_FILENO); // Standard Error  (FD 2)
    if (dev_null > 2) {
        close(dev_null);
    }
}

/* 8. Initialize System Logging */
openlog("firstdaemon", LOG_PID, LOG_DAEMON);
```

```
}

int main(void) {
   // Initialize the daemon initialization procedure
   skeleton_daemon();

   syslog(LOG_NOTICE, "First daemon started successfully.");

   /* Main Daemon Execution Loop */
   while (keep_running) {
       // TODO: Insert recurring daemon background tasks here
       syslog(LOG_INFO, "Daemon is running background task...");

       sleep(20);
   }

   syslog(LOG_NOTICE, "First daemon terminated gracefully.");
   closelog();

   return EXIT_SUCCESS;
}
```

When we run this source and check the result by the command: `ps aux`

```
khanhnp     9558  0.0  0.0   2776  1400 ?        S    09:54   0:00 ./test
```

Now we can see this process is no longer belong to any TTY. Besides, if we check the process tree by command: `pstree -p`

```
─systemd(1469)─|
               ├─test(9558)
```

We can see the daemon now is children of systemd process. WE can also check the syslog of this daemon by command: `journalctl -t firstdaemon -f`

```
Thg 9 30 09:54:27 khanhnp-17ZD90N-VX50K firstdaemon[9558]: First daemon started successfully.
Thg 9 30 09:54:27 khanhnp-17ZD90N-VX50K firstdaemon[9558]: Daemon is running background
task...
Thg 9 30 09:54:47 khanhnp-17ZD90N-VX50K firstdaemon[9558]: Daemon is running background
task...
Thg 9 30 09:55:07 khanhnp-17ZD90N-VX50K firstdaemon[9558]: Daemon is running background
```

```
task...
```

Now I will list all ID of the daemon during the creation time:

```
Thg 9 30 10:14:13 khanhnp-17ZD90N-VX50K firstdaemon[10037]: Parent: pid = 10037, ppid = 8261,
sid = 8261
Thg 9 30 10:14:13 khanhnp-17ZD90N-VX50K firstdaemon[10038]: Fork 1: pid = 10038, ppid = 10037,
sid = 8261
Thg 9 30 10:14:13 khanhnp-17ZD90N-VX50K firstdaemon[10038]: Setsid: pid = 10038, ppid = 1469,
sid = 10038
Thg 9 30 10:14:13 khanhnp-17ZD90N-VX50K firstdaemon[10039]: Fork 2: pid = 10039, ppid = 10038,
sid = 10038
```

> We call the method above is traditional system V creation daemon method. Recently, we have a new way: New-style daemon. However, in this documents, I will not mention about this topic, To read about it, please read
>
> https://0pointer.de/public/systemd-man/daemon.html#New-Style%20Daemons

To end this section, I will write bout a minimal daemon:

> The idea of this daemon is watch the status of specific process. Use signal IPC to check if the process live or death, use shared memory to check the pid of target process and if process if hang after threshold

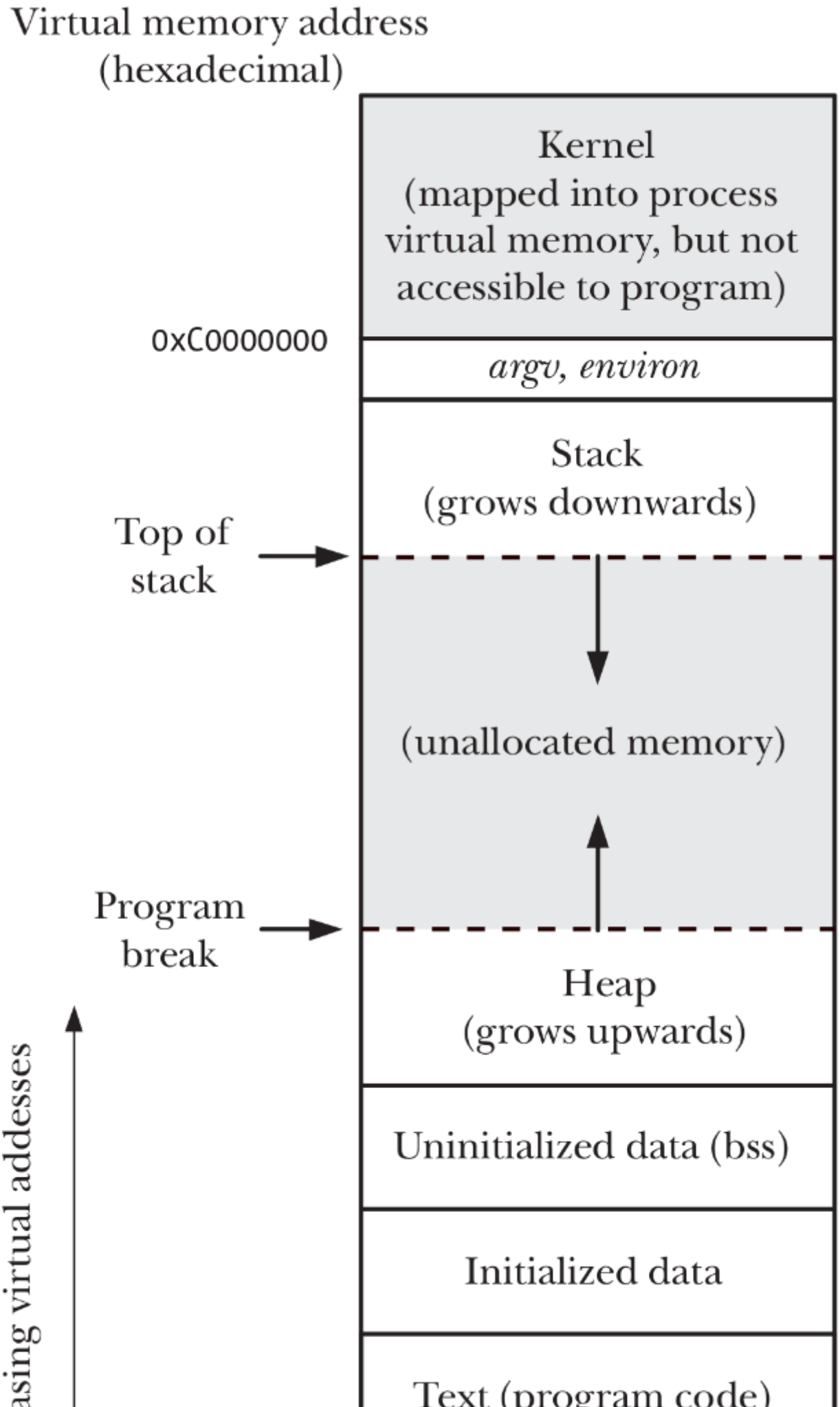
You can see the source in: https://github.com/220222112-PhamNgocKhanh/Zhone_trainee/tree/main/process_and_memory_layout/watchdog_daemon

2. Memory layout

Overview of memory layout

In the previous part, we understood about processs. Now we need know about the way that the process stored in the memory. Each process stored in memory by 5 main segments.

- **Text**: This segment contains the machine-language instructions of the program run by the process. This segment is **READ-ONLY**.
- **Initialized data**: This segment contains global and static variables that are explicitly initialized
- **Uninitialized data(BSS)**: This segment contains global and static variables that are not explicitly initialized or initialized with the value 0
- **Heap**: an area from which memory (for variables) can be dynamically allo- cated at run time
- **Stack**: a dynamically growing and shrinking segment containing stack frames. One stack frame is allocated for each currently called function. A frame stores the function’s local variables , arguments, and return value


Virtual memory address
(hexadecimal)
Kernel
(mapped into process
virtual memory, but not
accessible to program)
0xC0000000
argv, environ
Stack
(grows downwards)
Top of
stack
(unallocated memory)
Program
break
Heap
(grows upwards)
Uninitialized data (bss)
Initialized data
Text (program code)
Increasing virtual addesses

> In some cases, string literals and `const` data are stored in the `.rodata` section. However, other read-only data may be stored in the text segment because both sections are **read-only**.

The table below summarizes the different cases and the memory segment where each type of data is stored.

Type	Case	Scope	Typical segment
Normal	`int a = 5;` global	file/global	`.data`
Normal	`int a;` global	file/global	`.bss`
Normal	`int a = 5;` local	block	stack
Normal	`int a;` local	block	stack
`static`	`static int a = 5;` global	file	`.data`
`static`	`static int a;` global	file	`.bss`
`static`	`static int a = 5;` local	block	`.data`
`static`	`static int a;` local	block	`.bss`
`const`	`const int a = 5;` global	file	`.rodata`
`const`	`const int a = 5;` local	block	stack
`static const`	`static const int a = 5;` global	file	`.rodata`
`static const`	`static const int a = 5;` local	block	`.rodata`
String literal	`"hello"`	—	`.rodata`
Dynamic	`malloc(...)`	—	heap

> As we know, resources of every computers is not infinity, so we need to know the limit of any resources and the way to manage them.

This time to use the `getlimit` function and `setlimit` function.

- The `getrlimit` can show the resource that each process can be used like the maximum of stack, text segment, file descriptors, bytes in size.
- The `setrlimit` can set all field that we can see in the `getrlimit` function

An example:

```
int main()
{
    struct rlimit limit;
    getrlimit(RLIMIT_STACK, &limit);

    printf("soft limit of stack: %lu\n", limit.rlim_cur);
    printf("hard limit of stack: %lu\n", limit.rlim_max);

    printf("Stack soft limit: %.2f MB\n",
           (double)limit.rlim_cur / (1024 * 1024));

    printf("Stack hard limit: %.2f MB\n",
           (double)limit.rlim_max / (1024 * 1024));

    limit.rlim_cur = 16388608;



    setrlimit(RLIMIT_STACK, &limit);
    printf("soft limit of stack: %lu\n", limit.rlim_cur);
    printf("hard limit of stack: %lu\n", limit.rlim_max);

    printf("Stack soft limit: %.2f MB\n",
           (double)limit.rlim_cur / (1024 * 1024));

    printf("Stack hard limit: %.2f MB\n",
           (double)limit.rlim_max / (1024 * 1024));
}
```

The result look like:

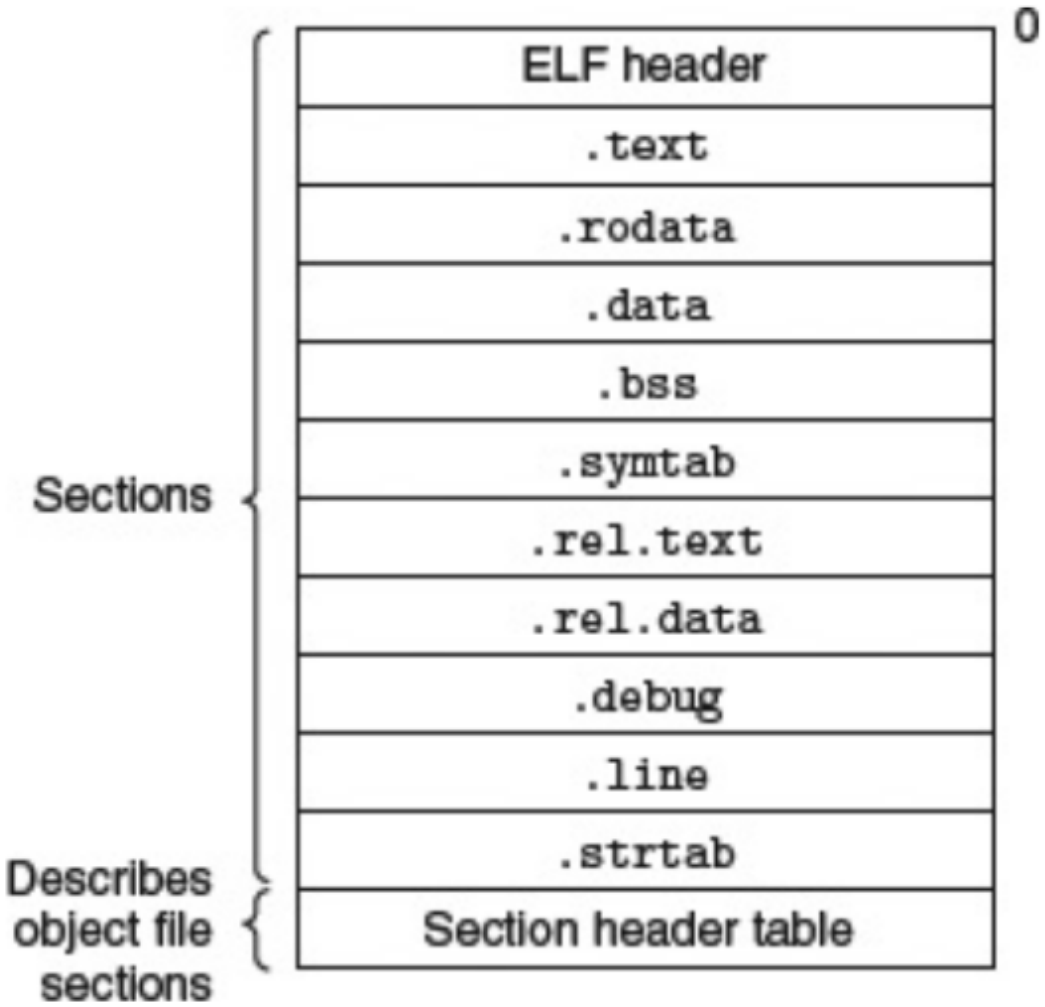
```
soft limit of stack: 8388608
hard limit of stack: 18446744073709551615
Stack soft limit: 8.00 MB
Stack hard limit: 17592186044416.00 MB
soft limit of stack: 16388608
hard limit of stack: 18446744073709551615
Stack soft limit: 15.63 MB
Stack hard limit: 17592186044416.00 MB
```

> Soft limit is the maximum stack resource that this process can use. Hard limit is the maximum satck resource that the system can allocate for this process.

Now, In the past documents, `C Programming review`, we known the execute of a program, after 4 stages, the execute file will be genarated. However, what is exactly inside this file? We can understand in by reading the following section: **`ELF Format, Symbol table and Relocation`**

ELF Format, Symbol table and Relocation

The execute file is presented by ELF format. We can see the structure of this format in this figure:



- The `.text` section contains the machine code of compied program
- The `.rodata` section contains `READ-ONLY` data like const, String literal
- The `.data` section contains initialized global and static C variables
- The `.bss` section contains uninitialized global and static C variables, along with any global or static variables that are initialized to zero

- The `.symtab`: A symbol table with information about functions and global variables that are defined and referenced in the program
- The `.rel.text`: A list of locations in the `.text` section that will need to be modified when the linker combines this object file with others
- `.rel.data` Relocation information for any global variables that are referenced or defined by the module
- `.debug`: A debugging symbol table with entries for local variables and typedefs defined in the program, global variables defined and referenced in the program, and the original C source file
- `.line` : A mapping between line numbers in the original C source program and machine code instructions in the `.text` section
- `.strtab` : A string table for the symbol tables in the .symtab and .debug sections and for the section names in the section headers

In a program, compiler/ linker need know about some crucial information like

- function
- global variable
- static variable
- external function/variable

We call them: `symbol`. A `symbol table` is a datastructure which is created by the compiler in compilation phase. it stores the mapping of symbol name to its corresponding memory address, type, size and scope. We can see all element of `symbol table` in this struct:

```
typedef struct
{
    int name;       /* String table offset */
    char type:4,    /* Function or data (4 bits) */
    char binding:4; /* Local or global (4 bits) */
    char reserved;  /* Unused */
    short section;  /* Section header index */
    long value;     /* Section offset or absolute address */
    long size;      /* Object size in bytes */
} Elf64_Symbol;
```

Look an example:

```
int num;
int f() {

    static int x = 5;
    return x;
```

```
}

int g() {
    static int x = 6;
    x++;
    return x;
}

int main()
{
}
```

After building this program, we can view the symbol table using the following command: `readelf -s` + `execfile`.

The important field that we need to pay attention to in the result should look like this:

Num	Value	Size	Type	Bind	Vis	Ndx	Name
12	`0x4010`	4	OBJECT	LOCAL	DEFAULT	23	`x.1`
13	`0x4014`	4	OBJECT	LOCAL	DEFAULT	23	`x.0`
20	`0x401c`	4	OBJECT	GLOBAL	DEFAULT	24	`num`
26	`0x1129`	16	FUNC	GLOBAL	DEFAULT	14	`f`
33	`0x1139`	31	FUNC	GLOBAL	DEFAULT	14	`g`
35	`0x1158`	15	FUNC	GLOBAL	DEFAULT	14	`main`

As we can see, the symbol table will list all important data in the program that the linker/compiler need know like the scope(bind), type, size, name and the value of them.

> **How Linkers Resolve Duplicate Symbol Names?**

Linker will use the following rules for dealing with duplicate symbol names:

- Rule 1. Multiple strong symbols with the same name are not allowed.
- Rule 2. Given a strong symbol and multiple weak symbols with the same name, choose the strong symbol.
- Rule 3. Given multiple weak symbols with the same name, choose any of the weak symbols.

Example of rule 1:

```
/* test.c */
 int x = 6;

int main()
{
printf("%d",x);
}

/* test2.c */
int x = 35;
```

> The linker will alert the **multiple definition** error

Example of rule 2:

```
/* test.c */
void func()
{
    printf("strong function");
}
int main()
{
    func();
}

/* test2.c */
__attribute__((weak)) void func()
{
    printf("weak function");
}
```

> The linker will choose the strong symbol function. In real situation, we can define more weak function in library, and if we need this function with another action, we can overwrite by strong function.

Example of rule 3:

```
/* test.c */
__attribute__((weak)) int x = 6;

int main()
{
printf("%d",x);
}

/* test2.c */
__attribute__((weak)) int x = 35;
```

> We can't know the result of this program, maybe `6` or `35`. Besides, if 2 variable have different type,The linker wiil choose the larger type.

Now we know every symbol in symbol table need a address to run. However, Compiler do not the exactly address of some variable, function that use in other files. So in linking process, linker need fill it in the symbol table. **Relocation** is the process linker do that.

We can see in the following example:

```
/* test.c */
extern int x;

int main()
{
    printf("%d\n", x);
    return 0;
}
/* test2.c */
int x = 5;
```

We run the command `gcc -o test.o -c test.c` and `gcc -o test2.o -c test2.c` to see the symbol table before linking. Now run `readelf -s test.o` :

```
Symbol table '.symtab' contains 7 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
     1: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS test.c
     2: 0000000000000000     0 SECTION LOCAL  DEFAULT    1 .text
```

```
     3: 0000000000000000     0 SECTION LOCAL  DEFAULT    5 .rodata
     4: 0000000000000000    43 FUNC    GLOBAL DEFAULT    1 main
     5: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND x
     6: 0000000000000000     0 NOTYPE  GLOBAL DEFAULT  UND printf
```

As we can see, both variable `x` and function `printf` don't have the address, now continue the linking process by command: `gcc -o test test.o test2.o` and run the command `readelf -s test` to see some special information:

```
Symbol table '.symtab' contains 38 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
    11: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS test.c
    12: 0000000000000000     0 FILE    LOCAL  DEFAULT  ABS test2.c
    23: 0000000000004010     4 OBJECT  GLOBAL DEFAULT   25 x
    25: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND printf@GLIBC_2.2.5
    33: 0000000000001149    43 FUNC    GLOBAL DEFAULT   16 main
```

Now we can see `x` symbol have the address after linker found the defintion of `x` in `test2.c`. However, what about `printf` function. To understand where the `printf` stay, we need understand the static and dynamic linking in the next section.

Static and dynamic linking

As we know in the previous section, the linker will fill all symbols that were defined in other files. We call them: **static linking**. The library source for this linking is **static library**

An example:

```
/* test.c */
int add(int a, int b);
int main()
{
    printf("%d", add(5, 6));
}

/* test2.c */
int add(int a, int b)
{
    return a + b;
}
```

We build the static library with two commands: `gcc -o add.o -c add.c` and `ar rcs libadd.a add.o`. After that, we can build the executable file using the command: `gcc test.c -L. -ladd -o test`.

Now, let's come back to the last section. The `printf` function is not defined in any of our source files, so why can the program still run?

We call this situation **dynamic linking**, and the library used for this is a **shared library**.

So, how are the symbol addresses that the linker does not fill in during the linking process resolved? The **dynamic linker** does this at **runtime**, not the compiler, as in static linking.

With the previous example, we can use the following command to create the shared library:

```
gcc -shared -o libadd.so add.o
```

Then, we can build the executable using:

```
gcc -o test_dynamic test.c ./libadd.so
```

The differrent of static and dynamic linking

Aspect	Static Library (`.a`)	Dynamic Library (`.so`)
File extension	`.a`	`.so`
Linking time	Mostly resolved at **build/link time**	Partly resolved at **load/run time**
Code location	Library code is copied into the executable when needed	Library code remains in the `.so` file
Executable size	Usually larger	Usually smaller
Runtime dependency	Does not need the `.a` file to run	Usually needs the `.so` at runtime
Memory usage	Each executable has its own copy of library code	The same `.so` code can be shared by multiple processes
Updating the library	Rebuild the executable	Often can replace/update the `.so` without rebuilding the executable, provided the ABI remains compatible
Symbol resolution	Mainly performed by the linker	Dynamic linker resolves symbols at runtime
Relocation	Mostly handled during static linking	Additional runtime relocations may be required

Stack memory layout

There are some defintion that we need to remember:

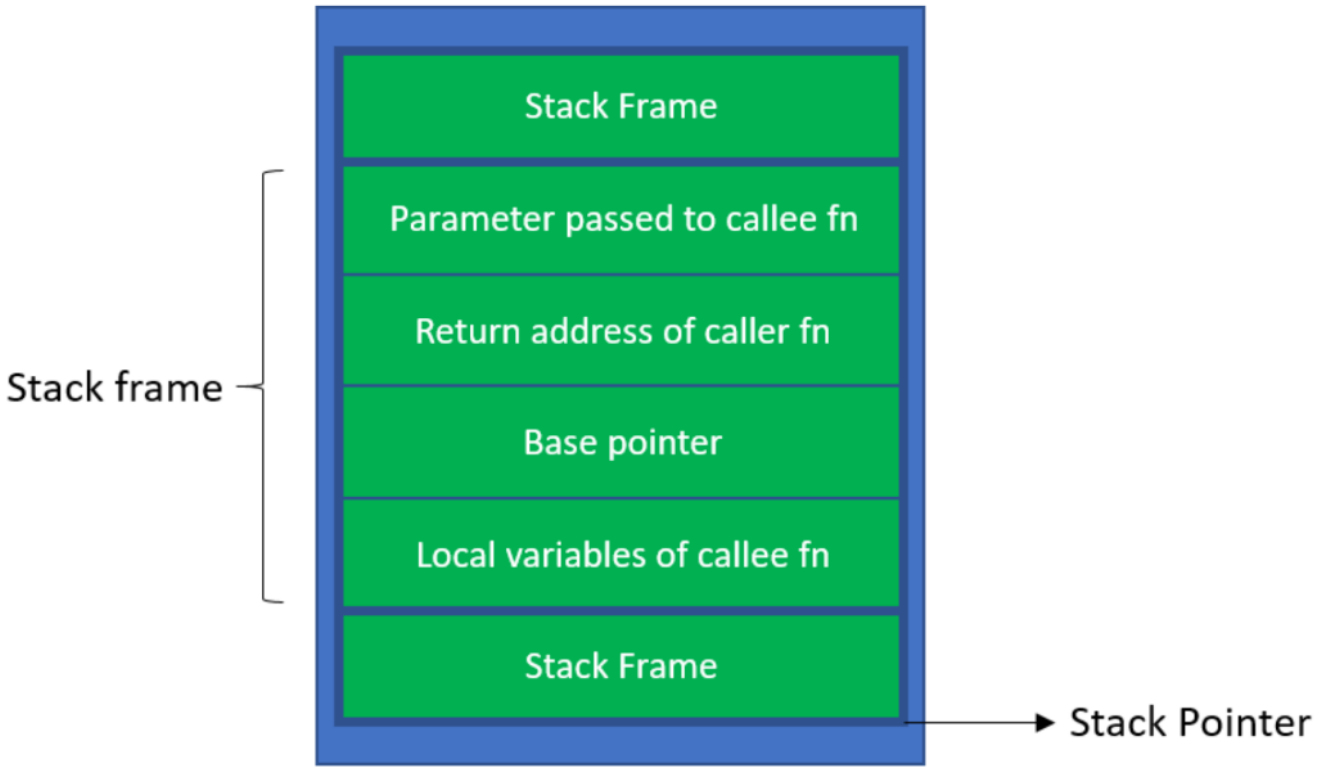
- **Caller function:** The function that contains the call to another function.

- **Callee function:** The function that is called by the caller.
- **Return address:** This field contains the address of the instruction immediately after the `call` instruction. For example:

```
0x401150:    call 0x401200
0x401155:    movl $10, -4(%rbp)
```

In the caller function, when execution reaches address `0x401150`, it jumps to the callee function. Therefore, the **return address** stored in the callee will be `0x401155`, so execution can return to the instruction immediately after the `call`.

- **RSP (Stack Pointer):** The register that stores the address of the top of the stack. This register changes as the stack grows or shrinks.
- **RBP (Frame Pointer):** The register that stores an address used as a reference point for the current stack frame. By using this pointer, we can access other locations within the stack frame using positive or negative offsets.



Firstly , we need to know, before every the function call, The arguments of this function will stored by **6 registers first**. The first is placed in `rdi`, the second in `rsi`, the third in `rdx`, and then `rcx`, `r8` and `r9`.

Now we go to an example to understand what happen in stack when the program run:

```
long multiply_add(int a, int b, int c)
{
    long x = a * b;
```

```
    long y = x + c;
    return y;
}

long calculate(int a, int b, int c, int d, int e, int f, int g)
{
    long x = multiply_add(a, b, c);
    long y = x + d + e;
    return y + f + g;
}

int main()
{
    long result = calculate(2, 3, 4, 5, 6, 7, 8);

    printf("result = %ld\n", result);

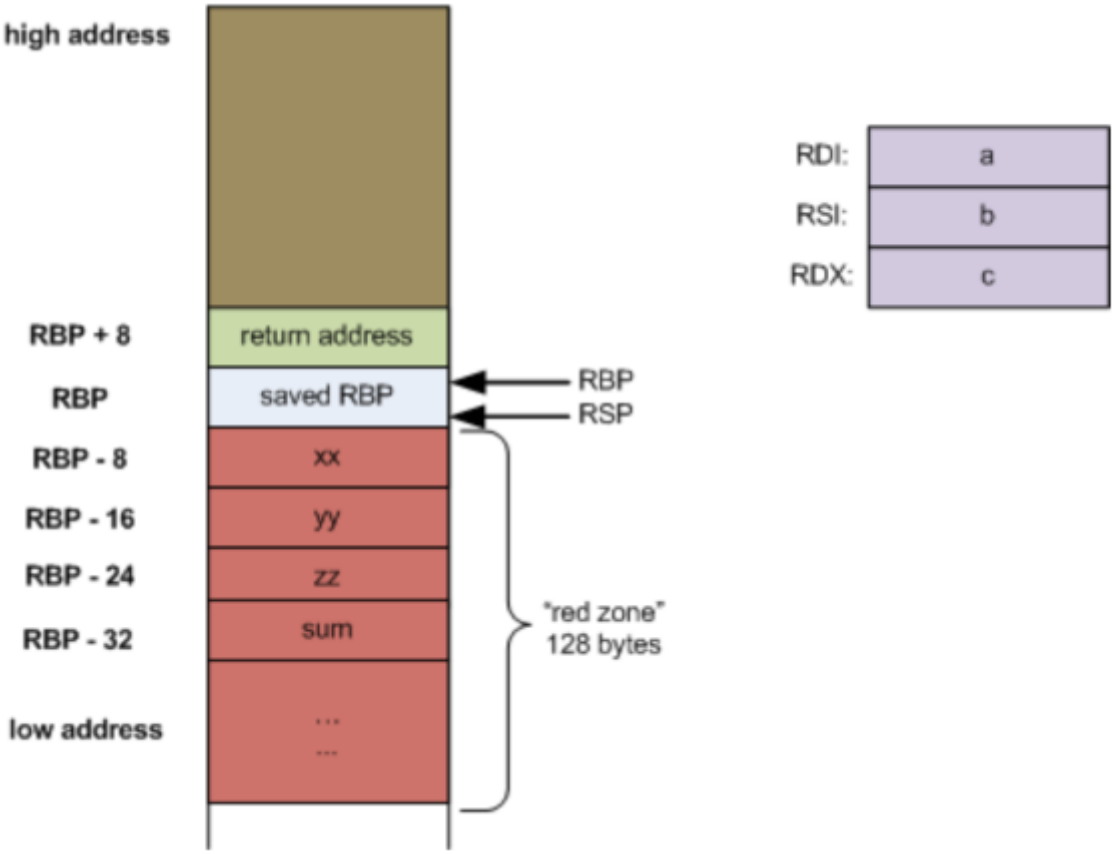
    return 0;
}
```

So, we export the asembly code of this program by `gcc -S test.c -o test.s`, i just take some important information inside:

```
multiply_add:
□pushq□%rbp
□movq□%rsp, %rbp
□movl□%edi, -20(%rbp)
□movl□%esi, -24(%rbp)
□movl□%edx, -28(%rbp)
□popq□%rbp
calculate:
□pushq□%rbp  /* store the caller RBP to the stack */
□movq□%rsp, %rbp /* change the RBP to the RSP register value */
□subq□$40, %rsp
□movl□%edi, -20(%rbp)  /* store value from 6 registers to stack */
□movl□%esi, -24(%rbp)
□movl□%edx, -28(%rbp)
□movl□%ecx, -32(%rbp)
□movl□%r8d, -36(%rbp)
□movl□%r9d, -40(%rbp)
```

```
	call	multiply_add
	movq	%rax, -16(%rbp)
	movl	-32(%rbp), %eax
	movslq	%eax, %rdx
	movq	-16(%rbp), %rax
	addq	%rax, %rdx
	movl	-36(%rbp), %eax
	addq	%rdx, %rax
	movq	%rax, -8(%rbp)
	movl	-40(%rbp), %eax
	movslq	%eax, %rdx
	movq	-8(%rbp), %rax
	addq	%rax, %rdx
	movl	16(%rbp), %eax
	leave
main:
.LFB8:
	pushq	%rbp
	movq	%rsp, %rbp
	subq	$16, %rsp
	pushq	$8
	movl	$7, %r9d /* store the value of arguments to 6 registers */
	movl	$6, %r8d
	movl	$5, %ecx
	movl	$4, %edx
	movl	$3, %esi
	movl	$2, %edi
	call	calculate
	addq	$8, %rsp
	movq	%rax, -8(%rbp)
	movq	-8(%rbp), %rax
	movq	%rax, %rsi
	leaq	.LC0(%rip), %rax
	movq	%rax, %rdi
	movl	$0, %eax
	call	printf@PLT
	movl	$0, %eax
	leave
```

An interesting information is in leaf function(that mean a function that doesn't call any other function), this function can use 128 bytes lower without change the RSP register like normal instuction. We call this area: **red zone**



> To deeper understand about this topic, go the lab at the end of documents, i will explain by using both `gdb` tool and assembly code

Lab practice

https://github.com/220222112-PhamNgocKhanh/Zhone_trainee/tree/main/process_and_memory_layout

---

Revision #18
Created 2026-08-21 02:24:08 UTC by Khanh Pham
Updated 2026-09-30 04:03:23 UTC by Khanh Pham